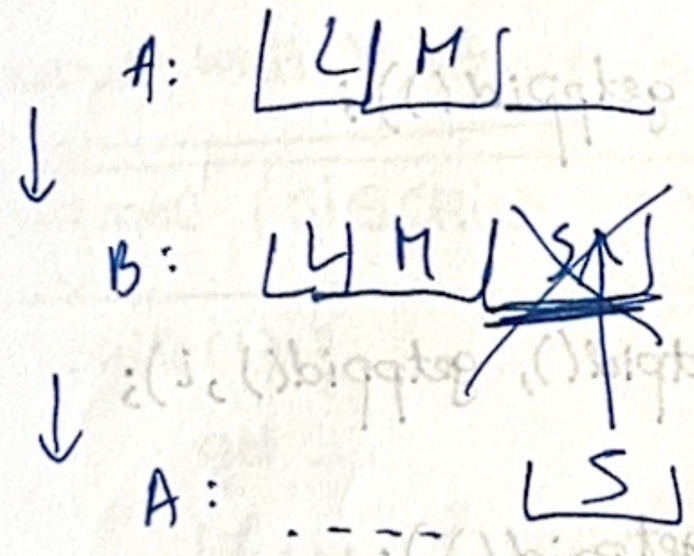
Threads problems

Lecture 4 - OS

18 mar. 2026.

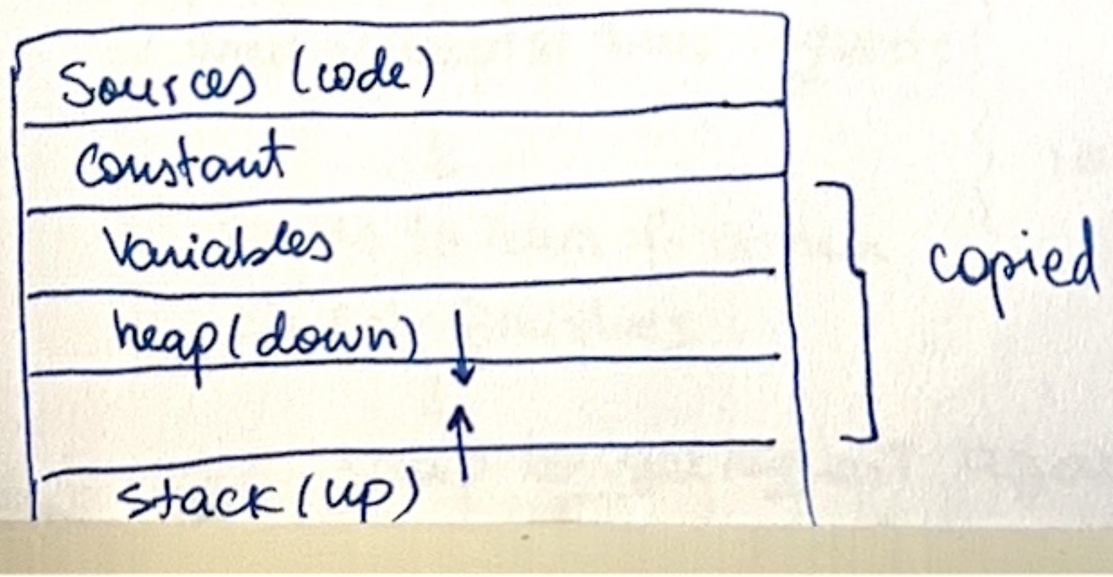
- Steps made by computer to do `n++`
  - L: load n from memory into register (CPU)
  - M: modify : increment value in that register.
  - S: store : save value back to memory

- RACE CONDITION: when program A does L, M, but does not finish S, program B does L, M, S and after A resumes, it saves its own version by overwriting B's work.



- forK()
  - → create new process = duplicate existing process using forK()
    - (CHILD) ← its own stack.
    - (PARENT)
  - clone
  - fork()
  - return 0
  - original
  - fork()
  - return PID of child.

- PROCESSES IN MEMORY:



```c
#include <stdio.h>
#include <unistd.h>
int main (int argc , char ** argv) {
        printf ("a %d %d \n", getpid(), getppid());
        fork(); //duplicate of current process.
        printf ("b %d %d \n" , getpid() , getppid());
        (void) argc;
        (void) argv;
        return 0;
```

process ID of current process

parent process id.

---

problematic :)

```c
#include <stdio.h>
#include <unistd.h>
int main (int argc, char ** argv) {
        printf ("a %d %d \n", getpid(), getppid());
        for( int i=0; i<3; i++) {
            fork();
            printf("c %d %d %d \n", getpid(), getppid(),i);
        }
        printf ("b %d %d \n" , getpid(), getppid());
        (void)argc;
        (void) argv;
        return 0;
}
```

```c
.....        {
printf ( a... );
for(int i=0; i<3; i++) {
    if (fork()==0){
        printf (c.. );
        exit(0);
    }
}
printf ("b ... );
(void) argc;
(void) argv;
return 0;
}
```

Handle Zombie Processes while maintaining high-performance server:

1.
```
while (1) {
    get request
    if (fork() == 0) {
        process
        respond
        exit(0)
    }
    wait(0);
```
→ defeats whole point of fork()

↳ parent process pauses. (it can't get new request until current one is finished)
↳ the parent will stay here until the child finishes, won't get any new request.

2. move wait() into a signal handler (f).

```
signal (SIGCHILD, f)
```
→ whenever child dies, don't wait; instead, jump to function f immediately.

```
while(1) {
    get ...
    if ... {
        process
        respond
        exit(0)
    }
}
```

never →
```
signal (SIGCHILD, SIG_IGN);
```
↳ parent not interested in child's exit status
↓
cleanest way to prevent zombies.

3.
```
void f (int sgm) {
    wait(0);
}
```
→ when child exits, it becomes zombie - it stays in the system table so parent can read its exit status
↓
wait(0)
↓
remove child from table.

↳ PRO: parent stays in while(1) and accepts new requests.
↓
pauses to run f when child finishes
↓
jump to get request again

signals.c

```
#
#
void f(int sgm) {
    printf("You wish...\n");
    (void)sgm;
}
int main(int argc, char** argv) {
    signal(SIGINT, f);
    while(1);
    (void) argc;
    (void) argv;
    return 0;
}
```

number of the signal that triggered the function.
(SIGINT → 2).

"callback function"
{
by OS

signal(SIGINT, f); → signal sent to a process when CTRL+C.
→ kills program

don't kill me
↓
pause whatever
and run f instead.